C++ Codebook

本 Codebook 收錄程式競賽常用的演算法與工具模板。

1. Basic Utilities

IO Speedup (cin, cout)

```
cin.sync_with_stdio(0);
cin.tie(0);
```

Buffered Output (Defer Flushing Until Program End)

```
ios::sync_with_stdio(false);
cin.tie(NULL);
```

Test With File

```
g++ main.cpp -o main
main < input.in > output.out
```

Write File

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ofstream outFile("example.txt");
    if (!outFile.is_open()) {
        std::cerr << "Error opening file for writing!" << std::endl;
        return 1;
    }
    outFile << "Hello, World!\n";
    outFile << "This is a line in a text file.\n";
    outFile << 123 << " " << 45.67 << "\n";
    outFile.close();
    return 0;
}
```

Read File

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ifstream inFile("example.txt");
    if (!inFile.is_open()) {
        std::cerr << "Error opening file for reading!" << std::endl;
        return 1;
    }
    std::string line;
    while (std::getline(inFile, line)) {
        std::cout << line << std::endl;
    }
    inFile.close();
    return 0;
}
```

Greatest Common Divisor (GCD)

```
int GCD(int x, int y) {
    if (!x) return y;
    if (!y) return x;
    while ((x %= y) && (y %= x));
    return x + y;
}
```

Least Common Multiple (LCM)

```
int LCM(int num1, int num2) {
    return ((num1 * num2) / GCD(num1, num2));
}
```

GCD and LCM in C++17

```
#include <numeric>
int main () {
    int a = 48, b = 18;
    int gcd = std::gcd(a, b);
    int lcm = std::lcm(a, b);
    return 0;
}
```

Priority Queue (std::priority_queue)

> **說明：** 示範 C++ 內建 `std::priority_queue`（位於 `<queue>`）的最大堆、最小堆與自訂結構排序。 - **預設：** 最大堆，最大的元素位於頂端。 - **最小堆：** 使用 `std::greater`，最小的元素位於頂端；圖論常搭配 `std::pair`。 - **自訂排序：** 在自訂 `struct` 中重載 `<` 運算子。

```
#include <queue>
#include <vector>
#include <iostream>
#include <utility> // for pair

using namespace std;

// Struct with custom sorting rules
struct Item {
    int id;
    int priority;

    // Custom sorting rule: sort by priority ascending (min-heap)
    // If priority is equal, sort by id ascending (smaller id at the top)
    // Note: Since priority_queue is a Max-Heap by default, we invert the comparison logic.
    bool operator<(const Item& other) const {
        if (priority != other.priority) {
            return priority > other.priority; // Larger priority means "less" priority in max-heap (so smaller comes to top)
        }
        return id > other.id; // Larger ID means "less" priority (so smaller ID comes to top)
    }
};

void priorityQueueUsage() {
    priority_queue<int> max_pq;
    max_pq.push(10);
    max_pq.push(30);
    max_pq.push(20);
    // max_pq.top() is 30 (largest element first)

    priority_queue<int, vector<int>, greater<int>> min_pq;
    min_pq.push(10);
    min_pq.push(30);
    min_pq.push(20);
```

```
    // min_pq.top() is 10 (smallest element first)

    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pair_pq;
    pair_pq.push({5, 101}); // {distance, node}
    pair_pq.push({2, 102});
    pair_pq.push({5, 100});
    // pair_pq.top() is {2, 102} (smallest distance first, then smallest node index)

    priority_queue<Item> custom_pq;
    custom_pq.push({101, 5});
    custom_pq.push({102, 5});
    custom_pq.push({103, 2});
    // custom_pq.top() is {103, 2} (priority 2), then {101, 5} (priority 5, smaller id first)
}
```

2. Prime Numbers

Count Primes (Sieve of Eratosthenes)

```
#include <vector>
#include <algorithm>

using namespace std;

int countPrimes(int N) {
    if (N < 2) return 0;
    vector<bool> isPrime(N + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= N; ++i) {
        if (isPrime[i]) {
            for (int j = i * i; j <= N; j += i) {
                isPrime[j] = false;
            }
        }
    }
    return count(isPrime.begin(), isPrime.end(), true);
}
```

Prime Number Test

```
bool isPrime(long long n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}
```

List Primes (Sieve of Eratosthenes)

```
#include <vector>
using namespace std;
vector<int> sieve(int N) {
    vector<bool> isPrime(N + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= N; ++i) {
        if (isPrime[i]) {
            for (int j = i * i; j <= N; j += i) {
                isPrime[j] = false;
            }
        }
    }
    vector<int> primes;
    for (int i = 2; i <= N; ++i) {
        if (isPrime[i]) {
            primes.push_back(i);
        }
    }
    return primes;
}
```

Prime Factorization

```
#include <vector>
#include <utility>
using namespace std;
vector<pair<long long, int>> primeFactorization(long long n) {
    vector<pair<long long, int>> factors;
    for (long long i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            int count = 0;
            while (n % i == 0) {
                count++;
                n /= i; // Eliminate the factor
            }
            factors.push_back({i, count});
        }
    }
    if (n > 1) {
        factors.push_back({n, 1}); // Remaining prime factor
    }
    return factors;
}
```

3. String Processing

String Split (Regex)

說明： 依照正規表示式指定的分隔符，將字串切成多個子字串。 **參數：** - `str`：要分割的字串。 - `keys`：分隔符的正規表示式，預設匹配連字號 `-` 或逗號 `,`。 - `keep`：若為 `true`，也會把匹配到的分隔符保留為獨立項目。 **用法：** 適合解析複雜格式的輸入，需引入 `<regex>`。

```
#include <vector>
#include <string>
#include <regex>

using namespace std;

vector<string> split(const string& str, string keys = "(-{1,3}|,)", bool keep = false) {
    vector<string> tokens;
    regex re(keys);
    sregex_token_iterator it;
    if (keep) {
        it = sregex_token_iterator(str.begin(), str.end(), re, {-1, 0});
    } else {
        it = sregex_token_iterator(str.begin(), str.end(), re, -1);
    }
    sregex_token_iterator end;
    while (it != end) {
        tokens.push_back(*it);
        ++it;
    }
    return tokens;
}
```

Knuth-Morris-Pratt (KMP) Substring Search

說明： 高效率字串匹配。先建立 LPS（最長相同真前後綴）表，跳過重複比較，無須回退文本索引。 **時間複雜度：** $O(N+M)$，N 為文本長度，M 為模式長度。 **空間複雜度：** $O(M)$，用於 LPS 表。 **用法：** 當暴力匹配的 $O(N \cdot M)$ 太慢時使用。

```cpp
#include <vector>
#include <string>

using namespace std;

// Build the Longest Prefix which is also a Suffix (LPS) table
vector<int> buildLPS(const string &pattern) {
    int m = pattern.size();
    vector<int> lps(m, 0);
    int len = 0; // Length of the previous longest prefix suffix
    for (int i = 1; i < m; ) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1]; // Backtrack in pattern
            } else {
                lps[i++] = 0;
            }
        }
    }
    return lps;
}

// KMP search main function: Returns starting indices of matches
vector<int> KMPsearch(const string &text, const string &pattern) {
    vector<int> result;
    int n = text.size(), m = pattern.size();
    if (m == 0) return result;
    vector<int> lps = buildLPS(pattern);
    int i = 0, j = 0; // i -> text index, j -> pattern index
    while (i < n) {
        if (text[i] == pattern[j]) {
            i++; j++;
            if (j == m) {
                result.push_back(i - j); // Found match at index (i - j)
                j = lps[j - 1];
            }
        } else {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return result;
}
```

4. Computational Geometry

Closest Pair of Points (Divide & Conquer)

說明： 使用分治法，找出二維平面中距離最近的兩點。 1. **排序：** 先依照 X 座標排序所有點。 2. **分治：** 分別遞迴求出左右半部的最短距離。 3. **合併：** 檢查中線兩側寬度 $2d$ 的帶狀區域，依 Y 座標排序，只比較 Y 座標差小於目前最短距離的點。 **時間複雜度：** 此實作每層排序帶狀區域，實際為 $O(N \log^2 N)$。 **空間複雜度：** $O(N)$。 **用法：** 適用最近點對問題；至少需輸入兩點。

```cpp
#include <vector>
#include <cmath>
#include <algorithm>
#include <limits>
#include <utility>

using namespace std;

struct Point {
    double x, y;
};

// Calculate Euclidean distance between two points
double dist(const Point& a, const Point& b) {
    return hypot(a.x - b.x, a.y - b.y);
}

// Brute force method for 3 or fewer points
double bruteForce(vector<Point>& points, int left, int right, pair<Point, Point>& bestPair) {
    double minDist = numeric_limits<double>::max();
    for (int i = left; i <= right; ++i) {
        for (int j = i + 1; j <= right; ++j) {
            double d = dist(points[i], points[j]);
            if (d < minDist) {
                minDist = d;
                bestPair = {points[i], points[j]};
            }
        }
    }
    return minDist;
}

// Check the strip area for closer points crossing the divide boundary
double stripClosest(vector<Point>& strip, double d, pair<Point, Point>& bestPair) {
    double minDist = d;
    sort(strip.begin(), strip.end(), [](const Point& a, const Point& b) {
        return a.y < b.y;
    });
    for (size_t i = 0; i < strip.size(); ++i) {
        // Compare each point with next points inside the strip
        for (size_t j = i + 1; j < strip.size() && (strip[j].y - strip[i].y) < minDist; ++j) {
            double newDist = dist(strip[i], strip[j]);
            if (newDist < minDist) {
                minDist = newDist;
                bestPair = {strip[i], strip[j]};
            }
        }
    }
    return minDist;
}

// Recursive Divide & Conquer helper utility
double closestUtil(vector<Point>& pointsSortedByX, int left, int right, pair<Point, Point>& bestPair) {
    // Use brute force if points count is 3 or less
    if (right - left <= 3) {
```

```
        return bruteForce(pointsSortedByX, left, right, bestPair);
    }
    int mid = (left + right) / 2;
    Point midPoint = pointsSortedByX[mid];
    pair<Point, Point> leftPair, rightPair;
    double dl = closestUtil(pointsSortedByX, left, mid, leftPair);
    double dr = closestUtil(pointsSortedByX, mid + 1, right, rightPair);
    double d = dl;
    bestPair = leftPair;
    if (dr < dl) {
        d = dr;
        bestPair = rightPair;
    }
    // Gather points inside vertical strip of width 2d
    vector<Point> strip;
    for (int i = left; i <= right; ++i) {
        if (abs(pointsSortedByX[i].x - midPoint.x) < d) {
            strip.push_back(pointsSortedByX[i]);
        }
    }
    pair<Point, Point> stripPair;
    double dStrip = stripClosest(strip, d, stripPair);
    if (dStrip < d) {
        d = dStrip;
        bestPair = stripPair;
    }
    return d;
}

// Main entry function to find the closest pair of points
pair<Point, Point> findClosestPair(vector<Point>& points) {
    vector<Point> pointsSortedByX = points;
    sort(pointsSortedByX.begin(), pointsSortedByX.end(), [](const Point& a, const Point& b) {
        return a.x < b.x;
    });
    pair<Point, Point> bestPair;
    closestUtil(pointsSortedByX, 0, pointsSortedByX.size() - 1, bestPair);
    return bestPair;
}
```

5. Permutation Operations

Next Permutation (Array-based)

```
#include <vector>
#include <algorithm>

using namespace std;

// Returns the lexicographically next permutation of the array.
vector<int> nextPermutation(vector<int> nums) {
    int n = nums.size();
    int i = n - 2;
    // Find the first decreasing element from the right
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        i--;
    }
    if (i >= 0) {
        // Find the successor to nums[i] from the right
        int j = n - 1;
        while (nums[j] <= nums[i]) {
            j--;
        }
        swap(nums[i], nums[j]);
    }
    // Reverse the remaining ascending suffix
    reverse(nums.begin() + i + 1, nums.end());
    return nums;
}
```

Permutation Rank (Lexicographical Rank)

```
#include <vector>
#include <algorithm>

using namespace std;

// Binary Indexed Tree (Fenwick Tree) helper
struct FenwickTree {
    int n;
    vector<int> tree;
    FenwickTree(int n) : n(n), tree(n + 1, 0) {}
    void update(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
};

// Calculates the 1-based lexicographical rank of a permutation.
// Assumes unique elements. Returns rank modulo MOD (default is 1e9 + 7).
long long getPermutationRank(vector<int> nums, long long MOD = 1000000007) {
    int n = nums.size();
    if (n == 0) return 1;

    // Precompute factorials modulo MOD
    vector<long long> fact(n, 1);
    for (int i = 1; i < n; ++i) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }

    // Coordinate compression to map values to [1, n]
    vector<int> temp = nums;
    sort(temp.begin(), temp.end());
    vector<int> compressed(n);
    for (int i = 0; i < n; ++i) {
        compressed[i] = lower_bound(temp.begin(), temp.end(), nums[i]) - temp.begin() + 1;
    }

    FenwickTree bit(n);
    for (int i = 1; i <= n; ++i) {
        bit.update(i, 1);
    }

    long long rank = 1;
    for (int i = 0; i < n; ++i) {
        // Count how many unused elements to the right are smaller than compressed[i]
        int smallerCount = bit.query(compressed[i] - 1);
```

```
        rank = (rank + smallerCount * fact[n - 1 - i]) % MOD;
        // Mark current element as used
        bit.update(compressed[i], -1);
    }
    return rank;
}
```

Get Permutation by Rank

說明： 使用階乘進位制（Lehmer code），將排名與大小 N 還原為對應排列。**時間複雜度：** 線性選取為 $O(N^2)$；搭配 Fenwick Tree 可做到 $O(N \log N)$。**空間複雜度：** $O(N)$。**用法：** 將從 0 起算的排名，轉成 $\{0, 1, \ldots, N-1\}$ 的字典序排列。

```
#include <vector>
#include <algorithm>

using namespace std;

// Returns the permutation corresponding to the given 0-based rank.
// Elements are drawn from {0, 1, ..., n-1}.
vector<int> getPermutationByRank(int n, long long rank) {
    // Precompute factorials
    vector<long long> fact(n, 1);
    for (int i = 1; i < n; ++i) {
        fact[i] = fact[i - 1] * i;
    }

    // Available elements (sorted)
    vector<int> available;
    for (int i = 0; i < n; ++i) available.push_back(i);

    vector<int> perm;
    for (int i = n - 1; i >= 0; --i) {
        // Determine which available element goes to this position
        int idx = rank / fact[i];
        rank %= fact[i];
        perm.push_back(available[idx]);
        available.erase(available.begin() + idx);
    }
    return perm;
}
```

6. Math

Euler Totient Function

說明： 計算 Euler 函數 $\phi(x)$，表示 1 到 x 中與 x 互質的整數個數。**時間複雜度：** $O(\sqrt{x})$。**空間複雜度：** $O(1)$。**用法：** 常用於模逆元、Euler 定理等數論問題；$\phi(1) = 1$ 的特例見最後備忘。

```
#include <cmath>

int Phi(int x) {
    if (x < 2) return 0;
    int ret = x;
    int sq = sqrt(x);
    for (int p = 2; p <= sq; p++) {
        if (x % p == 0) {
            while (x % p == 0) x /= p;
            ret -= ret / p;
        }
        if (x == 1) break;
    }
    if (x > 1) ret -= ret / x;
    return ret;
}
```

Modular Power (Fast Exponentiation)

說明： 使用二進位快速冪（反覆平方）計算 $a^b \pmod p$。**時間複雜度：** $O(\log b)$。**空間複雜度：** $O(1)$。**用法：** 處理模數下的大次方，是數論題常用的基礎工具。

```
// Computes a^b mod p
long long modpow(long long a, long long b, long long p) {
    long long res = 1;
    a %= p;
    while (b > 0) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}
```

Modular Inverse

說明： 模質數 p 時，不能直接套用一般除法。若 a 不被 p 整除，依費馬小定理，a^{p-2} 就是 a 的模乘法逆元。因此可把除以 a 改為乘以逆元：

$$x \div a \equiv x \times a^{p-2} \pmod p$$

時間複雜度： $O(\log p)$。**空間複雜度：** $O(1)$。**依賴：** 上面的 `modpow`。

```
// Computes a^{-1} mod p (p must be prime)
long long modInverse(long long a, long long p) {
    return modpow(a, p - 2, p);
}
```

Combinations Modulo p (nCr mod p)

說明： 預先計算階乘與逆階乘，之後可在 $O(1)$ 回答 $\binom{n}{r} \pmod p$。**時間複雜度：** 預處理 $O(N)$，每次查詢 $O(1)$。**空間複雜度：** $O(N)$。**依賴：** 上面的 `modpow` 與 `modInverse`。

```
#include <vector>
using namespace std;

const int MAXN = 1e6 + 5;
const long long MOD = 1e9 + 7;

vector<long long> fact(MAXN), inv_fact(MAXN);

void precompute(int n) {
    fact[0] = 1;
    for (int i = 1; i <= n; ++i) {
        fact[i] = fact[i - 1] * i % MOD;
    }
    inv_fact[n] = modInverse(fact[n], MOD);
    for (int i = n - 1; i >= 0; --i) {
        inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD;
    }
}

// Returns C(n, r) mod MOD
long long C(int n, int r) {
```

```
    if (r < 0 || r > n) return 0;
    return fact[n] % MOD * inv_fact[r] % MOD *
     ↪  inv_fact[n - r] % MOD;
}
```

7. Range Queries

Difference Array

說明： 對差分陣列的區間邊界加減，即可批次進行 $[L, R]$ 加值；最後做前綴和還原陣列。 **回傳：** 所有更新完成後，各位置覆蓋次數的最小值。 **時間複雜度：** $O(M + N)$，M 為更新次數，N 為陣列大小。 **空間複雜度：** $O(N)$，用於差分陣列。 **用法：** 適合多次區間更新，最後才查詢整體結果的離線情境。

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

long long differenceArray() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long N, M;
    if (!(cin >> N >> M)) return -1;

    // Size N+2 to prevent R+1 from exceeding
     ↪  boundary
    vector<long long> diff(N + 2, 0LL);

    // Read interval [L, R] for each turret
    for (int i = 0; i < M; i++) {
        long long L, R;
        cin >> L >> R;
        // Difference array update: +1 at start L, -1
         ↪  at position after end R+1
        diff[L] += 1;
        if (R + 1 <= N) {
            diff[R + 1] -= 1;
        }
    }

    // Compute prefix-sum directly on diff array and
     ↪  track the minimum value
    long long cover = 0;          // Current
     ↪  cumulative coverage at the current position
    long long answer = LLONG_MAX; // Stores the
     ↪  minimum coverage value

    for (int pos = 1; pos <= N; pos++) {
        cover += diff[pos]; // diff[pos] indicates
 ↪  the change in coverage at pos
        // cover represents the actual coverage count
         ↪  at pos
        if (cover < answer) {
            answer = cover;
        }
    }

    // Under normal circumstances, answer is at least
     ↪  0.
    // Return the answer.
    return answer;
}
```

Fenwick Tree (Binary Indexed Tree)

說明： 支援單點加值、前綴和與區間和查詢。 **複雜度：** 更新與查詢皆為 $O(\log N)$，空間為 $O(N)$。

```
struct BIT {
    int n;
    vector<long long> bit;

    BIT(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void add(int idx, long long val) {
        while (idx <= n) {
            bit[idx] += val;
            idx += idx & -idx;
        }
    }

    long long sum(int idx) {
        long long res = 0;

        while (idx > 0) {
            res += bit[idx];
            idx -= idx & -idx;
        }

        return res;
    }

    long long query(int l, int r) {
        return sum(r) - sum(l - 1);
    }
};
```

注意： - lowbit(i) 表示索引 i 所管理的區間長度：

```
lowbit(x) = x & -x
```

- tree[i] 儲存該索引管理範圍的總和。

8. Graph Algorithms

Dijkstra's Algorithm (Single-Source Shortest Path)

```
#include <vector>
#include <queue>
#include <climits>

using namespace std;

const long long INF = LLONG_MAX;

// adj[u] = list of {v, weight} representing edge u
 ↪  -> v with given weight
vector<long long> dijkstra(int n,
 ↪  vector<vector<pair<int, long long>>>& adj, int
 ↪  src) {
    vector<long long> dist(n + 1, INF);
    dist[src] = 0;

    // {distance, node}, min-heap by distance
    priority_queue<pair<long long, int>,
 ↪  vector<pair<long long, int>>, greater<>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) continue; // Outdated entry,
         ↪  skip

        for (auto& [v, w] : adj[u]) {
```

```
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    return dist; // dist[i] = shortest distance from src to i (INF if unreachable)
}
```

建圖範例：

```
int n = 5;
vector<vector<pair<int, long long>>> adj(n + 1); // 1-indexed, size n+1

auto addEdge = [&](int u, int v, long long w) {
    adj[u].push_back({v, w});
    adj[v].push_back({u, w}); // remove this line if the graph is directed
};

addEdge(1, 2, 4);
addEdge(1, 3, 1);
addEdge(3, 2, 2);

vector<long long> dist = dijkstra(n, adj, 1);
// dist[2] = 3 (via 1 -> 3 -> 2)
```

Bellman-Ford Algorithm (Handles Negative Weights + Cycle Detection)

```
#include <vector>
#include <climits>

using namespace std;

const long long INF = LLONG_MAX;

struct Edge {
    int u, v;
    long long w;
};

// Returns {dist, hasNegativeCycle}
// dist[i] = shortest distance from src to i (INF if unreachable)
pair<vector<long long>, bool> bellmanFord(int n, vector<Edge>& edges, int src) {
    vector<long long> dist(n + 1, INF);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < n - 1; i++) {
        for (auto& e : edges) {
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
            }
        }
    }

    // Check for negative weight cycle: if any edge can still be relaxed
    bool hasNegativeCycle = false;
    for (auto& e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
            hasNegativeCycle = true;
            break;
        }
    }

    return {dist, hasNegativeCycle};
}
```

建圖範例：

```
int n = 5;
vector<Edge> edges;

// Just push every edge into a flat vector, no grouping needed
edges.push_back({1, 2, 4});
edges.push_back({1, 3, 1});
edges.push_back({3, 2, 2});
edges.push_back({2, 4, -3}); // negative weight is fine

auto [dist, hasNegCycle] = bellmanFord(n, edges, 1);

if (hasNegCycle) {
    // Negative cycle exists, shortest path is undefined
} else {
    // dist[4] = shortest distance
}
```

Floyd-Warshall Algorithm (All-Pairs Shortest Path)

```
#include <vector>
#include <climits>

using namespace std;

const long long INF = LLONG_MAX / 2; // Avoid overflow when adding two INF values

// dist[i][j] initial state should be set by caller:
// dist[i][i] = 0, dist[i][j] = edge weight if edge exists, else INF
// Returns true if graph has no negative cycle, false otherwise
bool floydWarshall(int n, vector<vector<long long>>& dist) {
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    // Check for negative cycle: if any dist[i][i] < 0
    for (int i = 1; i <= n; i++) {
        if (dist[i][i] < 0) return false; // Negative cycle detected
    }

    return true; // No negative cycle, dist is valid
}
```

建圖範例：

```cpp
int n = 4;
vector<vector<long long>> dist(n + 1, vector<long long>(n + 1, INF));

// Initialize the diagonal
for (int i = 1; i <= n; i++) dist[i][i] = 0;

// Add edges (directed graph example)
dist[1][2] = 3;
dist[2][3] = 1;
dist[1][3] = 8; // if both a direct edge and a shorter indirect path exist, the algorithm resolves it automatically
dist[3][4] = 2;

bool ok = floydWarshall(n, dist);

if (!ok) {
    // Negative cycle exists
} else {
    // dist[1][4] = 6 (via 1 -> 2 -> 3 -> 4 = 3+1+2)
}
```

Minimum Spanning Tree (Kruskal's Algorithm)

說明： 求連通無向帶權圖的最小生成樹。先依邊權遞增排序，再貪心加入不形成環的邊；以 DSU 維護連通狀態。 **時間複雜度：** $O(E \log E + E\alpha(V))$，E 為邊數、V 為點數，α 為反 Ackermann 函數。 **空間複雜度：** DSU 的父節點與秩陣列為 $O(V)$。 **用法：** 適合稀疏圖；預設頂點使用 1-based 編號。

```cpp
#include <vector>
#include <algorithm>

using namespace std;

// Representation of an edge in the graph
struct Edge {
    int u, v;    // u, v: The two endpoint vertices (nodes) connected by this edge
    long long w; // w: The weight (cost/distance) of this edge
    // Overload the < operator to sort edges by weight
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

// Disjoint Set Union (DSU) / Union-Find structure
struct DSU {
    vector<int> parent; // parent[i] stores the parent/leader of element i
    vector<int> rank;   // rank[i] stores the approximate depth of the tree rooted at i (for Union by Rank optimization)

    DSU(int n) { // n: The number of vertices/elements in the graph
        parent.resize(n + 1);
        rank.resize(n + 1, 0);
        for (int i = 0; i <= n; i++) {
            parent[i] = i; // Initially, every vertex is its own parent (disjoint set)
        }
    }

    // Finds the representative (root leader) of the set containing element i
    int find(int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent[i]); // Path compression optimization
    }

    // Merges the set containing element i with the set containing element j.
    // Returns true if they were in different sets and successfully merged; false if they were already in the same set.
    bool unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);
        if (root_i != root_j) {
            // Union by rank optimization: attach the smaller tree under the larger tree
            if (rank[root_i] < rank[root_j]) {
                parent[root_i] = root_j;
            } else if (rank[root_i] > rank[root_j]) {
                parent[root_j] = root_i;
            } else {
                parent[root_j] = root_i;
                rank[root_i]++;
            }
            return true; // Successfully merged
        }
        return false; // Already in the same set (connecting them would form a cycle)
    }
};

// Computes the MST weight and optionally stores the selected edges.
// n: Number of vertices (nodes) in the graph.
// edges: Input vector containing all edges in the graph (will be sorted in-place).
// mstEdges: Output vector to store the selected edges that form the MST.
// Returns the total MST weight, or -1 if the graph is disconnected (no MST exists).
long long kruskal(int n, vector<Edge>& edges, vector<Edge>& mstEdges) {
    mstEdges.clear();
    sort(edges.begin(), edges.end()); // Sort edges in ascending order of weight (greedy approach)

    DSU dsu(n); // Disjoint Set Union to detect cycles
    long long mst_weight = 0; // Accumulator for the total weight of the MST
    int edges_used = 0; // Counter for the number of edges added to the MST

    for (const auto& edge : edges) {
        // Try to connect the two endpoints of the current edge (edge.u and edge.v)
        if (dsu.unite(edge.u, edge.v)) {
            mst_weight += edge.w;
            mstEdges.push_back(edge);
            edges_used++;
            // A tree with n vertices always has exactly n - 1 edges
            if (edges_used == n - 1) {
                break;
            }
        }
    }

    if (edges_used == n - 1) {
        return mst_weight;
    }
```

```
    return -1; // Graph is not fully connected (could not select n - 1 edges)
}
```

Cycle Detection

說明： 以 DFS 判斷圖是否有環。有向圖維護遞迴堆疊狀態；無向圖則檢查已訪問鄰點與父節點。 **時間複雜度：** $O(V+E)$。 **空間複雜度：** $O(V)$。

```
#include <vector>
using namespace std;

// Directed graph cycle detection via DFS
struct CycleDetector {
    int n;
    vector<vector<int>> adj;
    vector<int> color; // 0 = unvisited, 1 = in-stack, 2 = done

    CycleDetector(int n) : n(n), adj(n), color(n, 0) {}

    void addEdge(int u, int v) { adj[u].push_back(v); }

    bool dfs(int u) {
        color[u] = 1;
        for (int v : adj[u]) {
            if (color[v] == 1) return true;  // Back edge -> cycle
            if (color[v] == 0 && dfs(v)) return true;
        }
        color[u] = 2;
        return false;
    }

    // Returns true if a cycle exists in the directed graph
    bool hasCycle() {
        for (int i = 0; i < n; ++i) {
            if (color[i] == 0 && dfs(i)) return true;
        }
        return false;
    }
};

int main() {
    CycleDetector graph(4);
    // Add edges
    graph.addEdge(0, 1);
    graph.addEdge(1, 2);
    graph.addEdge(2, 0); // Cycle 0 -> 1 -> 2 -> 0
    graph.addEdge(2, 3);

    if (graph.hasCycle()) {
        cout << "Graph contains a cycle!" << endl;
    } else {
        cout << "Graph does not contain a cycle." << endl;
    }

    return 0;
}
```

Topological Sort (Kahn's Algorithm)

說明： 使用 Kahn 演算法，依入度為有向無環圖（DAG）建立拓撲序。 **時間複雜度：** $O(V+E)$。 **空間複雜度：** $O(V+E)$。 **用法：** 回傳拓撲排序後的頂點；若結果點數小於 V，表示存在有向環。

```
#include <vector>
#include <queue>
using namespace std;

// Returns topological order of vertices 0..n-1, or empty vector if cycle detected
vector<int> topoSort(int n, vector<vector<int>>& adj) {
    vector<int> indegree(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) indegree[v]++;

    queue<int> q;
    for (int i = 0; i < n; ++i)
        if (indegree[i] == 0) q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            if (--indegree[v] == 0) q.push(v);
        }
    }
    return (int)order.size() == n ? order : vector<int>{}; // empty if cycle
}

int main() {
    int n = 4;
    vector<vector<int>> adj(n);

    // Add edges: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3
    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(3);

    vector<int> result = topoSort(n, adj);

    if (result.empty()) {
        cout << "Graph contains a cycle (No valid topological order)." << endl;
    } else {
        cout << "Topological Sort Order: ";
        for (int u : result) {
            cout << u << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Strongly Connected Components (Tarjan's Algorithm)

說明： 使用 Tarjan 的單次 DFS，找出有向圖的所有強連通分量（SCC）。 **時間複雜度：** $O(V+E)$。 **空間複雜度：** $O(V)$。 **用法：** 每個 SCC 是任意兩點皆互相可達的極大頂點集合，可用於縮點。

```
#include <vector>
#include <stack>
using namespace std;

struct Tarjan {
    int n, timer = 0;
    vector<vector<int>> adj;
    vector<int> disc, low, comp;
    vector<bool> onStack;
    stack<int> st;
```

```
    int sccCount = 0;

    Tarjan(int n) : n(n), adj(n), disc(n, -1), low(n), comp(n, -1), onStack(n, false) {}

    void addEdge(int u, int v) { adj[u].push_back(v); }

    void dfs(int u) {
        disc[u] = low[u] = timer++;
        st.push(u);
        onStack[u] = true;
        for (int v : adj[u]) {
            if (disc[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (onStack[v]) {
                low[u] = min(low[u], disc[v]);
            }
        }
        if (low[u] == disc[u]) {
            while (true) {
                int v = st.top(); st.pop();
                onStack[v] = false;
                comp[v] = sccCount;
                if (v == u) break;
            }
            sccCount++;
        }
    }

    // Returns comp[i] = SCC id of vertex i (0-indexed)
    int solve() {
        for (int i = 0; i < n; ++i)
            if (disc[i] == -1) dfs(i);
        return sccCount;
    }
};

int main() {
    // Graph: 0 -> 1 -> 2 -> 0 (one SCC), 2 -> 3 (3 is its own SCC)
    Tarjan tj(4);
    tj.addEdge(0, 1);
    tj.addEdge(1, 2);
    tj.addEdge(2, 0); // Cycle: forms SCC {0, 1, 2}
    tj.addEdge(2, 3); // 3 has no back edge, forms SCC {3}

    int numSCC = tj.solve();
    cout << "Number of SCCs: " << numSCC << endl; // 2

    // Print SCC id of each vertex
    for (int i = 0; i < 4; ++i) {
        cout << "Node " << i << " -> SCC " << tj.comp[i] << endl;
    }
    // Node 0, 1, 2 share the same SCC id; Node 3 has a different id

    return 0;
}
```

Disjoint Set Union (DSU / Union-Find)

說明： 支援路徑壓縮與依秩 / 大小合併的 DSU，可高效率維護互斥集合與連通關係。 **時間複雜度：** 每次操作攤銷 $O(\alpha(N))$，幾乎為 $O(1)$；α 為反 Ackermann 函數。 **空間複雜度：** $O(N)$。 **用法：** 連通分量、無向圖判環、Kruskal 最小生成樹等。

```
#include <vector>
using namespace std;

struct DSU {
    vector<int> parent, sz;

    DSU(int n) : parent(n + 1), sz(n + 1, 1) {
        for (int i = 0; i <= n; ++i) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // Path compression
        return parent[x];
    }

    // Returns true if x and y were in different sets (successfully merged)
    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y); // Union by size
        parent[y] = x;
        sz[x] += sz[y];
        return true;
    }

    bool connected(int x, int y) { return find(x) == find(y); }
};

int main() {
    DSU dsu(5);
    dsu.unite(0, 1);
    dsu.unite(1, 2);
    dsu.unite(3, 4);

    cout << dsu.connected(0, 2) << endl; // 1 (true: 0-1-2 are in the same set)
    cout << dsu.connected(0, 3) << endl; // 0 (false: {0,1,2} and {3,4} are separate)

    dsu.unite(2, 3);
    cout << dsu.connected(0, 4) << endl; // 1 (true: all merged now)

    return 0;
}
```

Lowest Common Ancestor (Binary Lifting)

說明： 預處理每個節點的 2^k 層祖先，使用倍增法查詢最近共同祖先（LCA）。 **時間複雜度：** 預處理 $O(N \log N)$，每次查詢 $O(\log N)$。 **空間複雜度：** $O(N \log N)$。 **用法：** 有根樹的共同祖先查詢，常搭配樹上路徑查詢與樹 DP。

```
#include <vector>
using namespace std;

const int MAXLOG = 18; // Supports up to 2^18 = 262144 nodes

struct LCA {
    int n, LOG;
    vector<int> depth;
    vector<vector<int>> up; // up[v][k] = 2^k-th ancestor of v
    vector<vector<int>> adj;

```

```
    LCA(int n) : n(n), LOG(MAXLOG), depth(n + 1, 0),
↪ up(n + 1, vector<int>(MAXLOG, 0)), adj(n + 1) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int u, int p) {
        up[u][0] = p;
        for (int k = 1; k < LOG; ++k)
            up[u][k] = up[up[u][k - 1]][k - 1];
        for (int v : adj[u]) {
            if (v != p) {
                depth[v] = depth[u] + 1;
                dfs(v, u);
            }
        }
    }

    void build(int root = 1) { dfs(root, root); }

    int query(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        int diff = depth[u] - depth[v];
        for (int k = 0; k < LOG; ++k)
            if ((diff >> k) & 1) u = up[u][k];
        if (u == v) return u;
        for (int k = LOG - 1; k >= 0; --k)
            if (up[u][k] != up[v][k]) { u = up[u]
↪  [k]; v = up[v][k]; }
        return up[u][0];
    }
};

int main() {
    // Tree structure (1-indexed):
    //       1
    //      / \
    //     2   3
    //    / \
    //   4   5
    LCA lca(5);
    lca.addEdge(1, 2);
    lca.addEdge(1, 3);
    lca.addEdge(2, 4);
    lca.addEdge(2, 5);
    lca.build(1); // Root at node 1

    cout << lca.query(4, 5) << endl; // 2 (LCA of 4
↪ and 5 is 2)
    cout << lca.query(4, 3) << endl; // 1 (LCA of 4
↪ and 3 is 1)
    cout << lca.query(2, 4) << endl; // 2 (LCA of 2
↪ and 4 is 2)

    return 0;
}
```

Bipartite Matching (Hungarian Algorithm)

求二分圖的最大匹配數。左側集合 U 編號為 0 到 $n-1$，右側集合 V 編號為 0 到 $m-1$。

```
#include <vector>

struct Hungarian {
    int n, m;
    std::vector<std::vector<int>> adj; // adj[u]
↪  contains neighbors in V
    std::vector<int> match_v;          // match_v[v]
↪  stores the matched u node
    std::vector<bool> visited;

    Hungarian(int n, int m) : n(n), m(m), adj(n),
↪ match_v(m, -1), visited(m, false) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    // DFS to find an augmenting path
    bool dfs(int u) {
        for (int v : adj[u]) {
            if (visited[v]) continue;
            visited[v] = true;

            // If v is not matched, or its match can
↪  find another choice
            if (match_v[v] == -1 || dfs(match_v[v]))
↪  {
                match_v[v] = u;
                return true;
            }
        }
        return false;
    }

    // Returns the maximum matching size
    int solve() {
        int ans = 0;
        std::fill(match_v.begin(), match_v.end(),
↪  -1);
        for (int i = 0; i < n; ++i) {
            std::fill(visited.begin(), visited.end(),
↪  false);
            if (dfs(i)) ans++;
        }
        return ans;
    }
};
```

9. Sequence Algorithms

Longest Increasing Subsequence (LIS)

求最長嚴格遞增子序列長度，時間 $O(N \log N)$、空間 $O(N)$。

```
#include <vector>
#include <algorithm>

// Returns the length of the LIS
int getLIS(const std::vector<int>& nums) {
    if (nums.empty()) return 0;

    std::vector<int> v;
    for (int x : nums) {
        auto it = std::lower_bound(v.begin(),
↪  v.end(), x);
        if (it == v.end()) {
            v.push_back(x); // x is larger than all
↪  elements, extend the sequence
        } else {
            *it = x; // Replace the first element >=
↪  x to maintain a smaller tail
        }
    }
    return v.size();
}
```

10. Trie (Prefix Tree)

說明： Trie（前綴樹）透過共用前綴儲存字串，支援插入、完整單字查找與前綴查詢。**時間複雜度：** insert、search、startsWith 皆為 $O(L)$，L 為字串長度。**空間複雜度：** $O(\Sigma L)$，ΣL 為所有插入字串的總長度。

```cpp
#include <vector>
#include <string>

using namespace std;

struct Trie {
    // Each Trie node stores:
    // nxt[i] : the index of the child corresponding to character ('a' + i)
    // isEnd  : true if a word ends at this node
    struct Node {
        int nxt[26];
        bool isEnd;

        Node() {
            fill(nxt, nxt + 26, -1);
            isEnd = false;
        }
    };

    vector<Node> trie;

    Trie() {
        trie.emplace_back(); // Node 0 is the root
    }

    // Inserts a string into the Trie.
    void insert(const string& word) {
        int current = 0;

        for (char ch : word) {
            int index = ch - 'a';

            // Create a new node if the path does not exist
            if (trie[current].nxt[index] == -1) {
                trie[current].nxt[index] = trie.size();
                trie.emplace_back();
            }

            current = trie[current].nxt[index];
        }

        trie[current].isEnd = true;
    }

    // Returns true if the exact word exists in the Trie.
    bool search(const string& word) {
        int current = 0;

        for (char ch : word) {
            int index = ch - 'a';

            if (trie[current].nxt[index] == -1)
                return false;

            current = trie[current].nxt[index];
        }

        return trie[current].isEnd;
    }

    // Returns true if there exists any word with the given prefix.
    bool startsWith(const string& prefix) {
        int current = 0;

        for (char ch : prefix) {
            int index = ch - 'a';

            if (trie[current].nxt[index] == -1)
                return false;

            current = trie[current].nxt[index];
        }

        return true;
    }
};
```

11. Aho-Corasick Automaton (AC Automaton)

說明： 在 Trie 上加入類似 KMP 的失敗連結，支援多模式字串匹配，可一次掃描文本找出多個模式。

適合使用 AC 自動機的情境：

- 在同一文本中匹配多個模式
- 敏感詞過濾
- 字典匹配
- DNA 序列匹配
- 字串 DP / 自動機 DP

若只有一個模式，通常使用 KMP 即可。

Algorithm Steps

模式集合 -> 建立 Trie -> 建立失敗連結（BFS）-> 掃描文本

失敗連結讓匹配從最長合法後綴繼續，不必重新從根節點開始。

```cpp
struct AhoCorasick {
    static constexpr int SIGMA = 26;

    struct Node {
        int nxt[SIGMA];
        int fail;
        vector<int> out;

        Node() {
            memset(nxt, 0, sizeof(nxt));
            fail = 0;
        }
    };

    vector<Node> trie;

    AhoCorasick() {
        trie.emplace_back();    // root
    }

    void insert(const string &s, int id) {
        int cur = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!trie[cur].nxt[x]) {
                trie[cur].nxt[x] = trie.size();
                trie.emplace_back();
            }
            cur = trie[cur].nxt[x];
        }
        trie[cur].out.push_back(id);
    }
```

```
    void build() {
        queue<int> q;

        for (int c = 0; c < SIGMA; c++) {
            int v = trie[0].nxt[c];
            if (v) q.push(v);
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int c = 0; c < SIGMA; c++) {
                int &v = trie[u].nxt[c];

                if (v) {
                    trie[v].fail =
↪   trie[trie[u].fail].nxt[c];
                    trie[v].out.insert(
                        trie[v].out.end(),

↪   trie[trie[v].fail].out.begin(),
                        trie[trie[v].fail].out.end()
                    );
                    q.push(v);
                } else {
                    v = trie[trie[u].fail].nxt[c];
                }
            }
        }
    }

    vector<int> query(const string &text) {
        vector<int> res;
        int cur = 0;

        for (char c : text) {
            cur = trie[cur].nxt[c - 'a'];
            for (int id : trie[cur].out)
                res.push_back(id);
        }

        return res;
    }
};
```

Procedure

1. 將所有模式插入 Trie。
2. 使用 BFS 建立失敗連結。
3. 掃描文本一次。
4. 回報所有匹配的模式。

Notes

- 根節點編號通常為 0。
- 使用 BFS 建立失敗連結。
- `build()` 會補齊缺少的轉移；掃描轉移為 $O(|Text|)$，輸出所有匹配另需加上輸出量。
- `out` 儲存以目前節點結尾的所有模式 ID。
- 常搭配 DP 解決禁止字串、字串計數等問題。

12. Expected Value DP

想法： 定義每個狀態的期望值，直接列出期望方程。

General Formula

$$E(s) = \sum P_i \left(C_i + E(next_i)\right)$$

其中：

- P_i：轉移發生的機率
- C_i：立即付出的成本
- $E(next_i)$：轉移後的未來期望值

If Self-loop Exists

假設：

$$E(s) = A + pE(s)$$

將自迴圈項移到等號左側：

$$E(s) = \frac{A}{1-p}$$

這是期望值 DP 最常見的移項方式。

Steps

1. 定義 DP 狀態。
2. 列舉所有可能轉移。
3. 直接列出期望方程。
4. 若有自迴圈，將對應項移到左側。
5. 解出方程。
6. 使用記憶化搜尋或 DP 計算。

Common State Types

- `dp[i]`
- `dp[mask]`
- `dp[x][y]`
- `dp[a][b][c]`

13. Advanced Data Structures

Segment Tree with Lazy Propagation

說明： 以延遲標記支援區間更新與區間查詢，將更新推遲到需要存取子節點時再處理。下列實作為區間加值 / 區間和。**時間複雜度：** 每次更新與查詢 $O(\log N)$。**空間複雜度：** $O(N)$。**用法：** 適合需要同時進行區間修改與區間查詢的題目。

```
#include <vector>
using namespace std;

// Segment Tree with Lazy Propagation (Range Add,
↪  Range Sum)
struct SegTree {
    int n;
    vector<long long> tree, lazy;

    SegTree(int n) : n(n), tree(4 * n, 0), lazy(4 *
↪  n, 0) {}

    void build(vector<int>& a, int node, int start,
↪  int end) {
        if (start == end) {
            tree[node] = a[start];
        } else {
            int mid = (start + end) / 2;
            build(a, 2 * node, start, mid);
            build(a, 2 * node + 1, mid + 1, end);
            tree[node] = tree[2 * node] + tree[2 *
↪  node + 1];
        }
    }

    void pushDown(int node, int start, int end) {
        if (lazy[node] != 0) {
            int mid = (start + end) / 2;
```

```
            tree[2 * node]     += lazy[node] * (mid - start + 1);
            tree[2 * node + 1] += lazy[node] * (end - mid);
            lazy[2 * node]     += lazy[node];
            lazy[2 * node + 1] += lazy[node];
            lazy[node] = 0;
        }
    }

    // Range update: add val to all elements in [l, r]
    void update(int node, int start, int end, int l, int r, long long val) {
        if (r < start || end < l) return;
        if (l <= start && end <= r) {
            tree[node] += val * (end - start + 1);
            lazy[node] += val;
            return;
        }
        pushDown(node, start, end);
        int mid = (start + end) / 2;
        update(2 * node, start, mid, l, r, val);
        update(2 * node + 1, mid + 1, end, l, r, val);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    // Range query: sum of elements in [l, r]
    long long query(int node, int start, int end, int l, int r) {
        if (r < start || end < l) return 0;
        if (l <= start && end <= r) return tree[node];
        pushDown(node, start, end);
        int mid = (start + end) / 2;
        return query(2 * node, start, mid, l, r) +
               query(2 * node + 1, mid + 1, end, l, r);
    }

    // Convenience wrappers (1-indexed)
    void update(int l, int r, long long val) { update(1, 1, n, l, r, val); }
    long long query(int l, int r) { return query(1, 1, n, l, r); }
};

int main() {
    // Array: [1, 2, 3, 4, 5] (1-indexed)
    vector<int> a = {0, 1, 2, 3, 4, 5}; // a[0] unused
    SegTree st(5);
    st.build(a, 1, 1, 5);

    cout << st.query(1, 5) << endl; // 15 (sum of all)
    cout << st.query(2, 4) << endl; // 9  (sum of a[2..4] = 2+3+4)

    st.update(2, 4, 10); // Add 10 to each element in [2, 4]
    // Array is now: [1, 12, 13, 14, 5]

    cout << st.query(1, 5) << endl; // 45 (15 + 10*3)
    cout << st.query(2, 4) << endl; // 39 (12+13+14)

    return 0;
}
```

14. Supplementary Templates --- Usage and Indexing

以下模板補充第 1--13 章。使用時貼在 `main` 前；原有章節與模板保留。新增模板預設使用 **0-based 索引與閉區間 [l,r]**。RangeBIT 使用 **1-based**；HLD 接原有線段樹時，索引需加 1。 先貼下列共用宣告，再貼所需模板與依賴。距離、總和、乘積使用 `ll`，並確保所有中間結果不溢位。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
const ll BOOK_INF = numeric_limits<ll>::max()/4;
using Graph01 = vector<vector<pair<int,ll>>>;
```

快速選用：無權最短路用 BFS，0/1 邊權用 0--1 BFS，一般非負權沿用第 8 章 Dijkstra。 靜態區間最小值用 Sparse Table；區間加值與區間和用 RangeBIT 或第 13 章 Lazy Segment Tree。 樹上路徑用 HLD，容量限制用 Dinic，布林限制用 2-SAT，小集合問題用 bitmask DP 或 MITM。

15. Additional Range and Sequence Techniques

Coordinate Compression

用途： 將大值域映射為連續編號，保留大小順序，適合搭配 BIT 或 Mo 演算法。 **用法：** `auto v=compress(a)`；x 的編號為 `lower_bound(v.begin(),v.end(),x)-v.begin()`。 接 BIT 時編號加 1。時間 $O(n \log n)$；不保留數值間的實際距離。

```
template<class T>
vector<T> compress(vector<T> a) {
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(),a.end()),a.end());
    return a;
}
```

Range Add / Range Sum with Two BITs

用法： `RangeBIT b(n); b.add(l,r,x); b.query(l,r)`。1-based，每次操作 $O(\log n)$。 **依賴：** 第 7 章 BIT。以 `add(i,i,a[i])` 建立初值；不可更新索引 0。

```
struct RangeBIT {
    int n; BIT b1,b2;
    RangeBIT(int n):n(n),b1(n),b2(n) {}
    void boundary(int i,ll x) {
        if (i>n) return;
        b1.add(i,x); b2.add(i,x*(i-1));
    }
    void add(int l,int r,ll x) {
        boundary(l,x); boundary(r+1,-x);
    }
    ll prefix(int x) {
        return x*b1.sum(x)-b2.sum(x);
    }
    ll query(int l,int r) {
        return prefix(r)-prefix(l-1);
    }
};
```

Inversion Count

用法： `inversions(a)` 計算 i < j 且 a[i] > a[j] 的配對數；相等不計。 **依賴：** `compress` 與第 7 章 BIT。時間 $O(n \log n)$，答案使用 `ll`。

```
ll inversions(const vector<ll>& a) {
    auto v=compress(a);
    BIT b(v.size()); ll ans=0;
    for (int i=0;i<(int)a.size();++i) {
        int x=lower_bound(v.begin(),v.end(),a[i])
            -v.begin()+1;
```

```
        ans+=i-b.sum(x); b.add(x,1);
    }
    return ans;
}
```

Sparse Table (Static Range Minimum)

用法： `SparseMin st(a); st.query(l,r)`。建立與空間 $O(n \log n)$，查詢 $O(1)$。僅適用不修改的陣列。可改 min/max/gcd；重疊區間合併**不能用來求和**。

```
struct SparseMin {
    vector<int> lg; vector<vector<ll>> st;
    SparseMin(const vector<ll>& a) {
        int n=a.size(); lg.resize(n+1);
        for (int i=2;i<=n;++i) lg[i]=lg[i/2]+1;
        st.assign(n?lg[n]+1:1,vector<ll>(n));
        st[0]=a;
        for (int k=1;k<(int)st.size();++k)
            for (int i=0;i+(1<<k)<=n;++i)
                st[k][i]=min(st[k-1][i],
                    st[k-1][i+(1<<(k-1))]);
    }
    ll query(int l,int r) const {
        int k=lg[r-l+1];
        return min(st[k][l],st[k][r-(1<<k)+1]);
    }
};
```

Monotonic Deque (Sliding Window Minimum)

用法： `windowMin(a,k)` 依序回傳每個長度 k 視窗的最小值，k >= 1。時間 $O(n)$。求最大值時，把 >= 改成 <=；佇列存索引，方便移除過期元素。

```
vector<ll> windowMin(const vector<ll>& a,int k) {
    assert(k>=1);
    deque<int> q; vector<ll> ans;
    for (int i=0;i<(int)a.size();++i) {
        while (!q.empty() && q.front()<=i-k)
            q.pop_front();
        while (!q.empty() && a[q.back()]>=a[i])
            q.pop_back();
        q.push_back(i);
        if (i+1>=k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

Monotonic Stack (Previous Smaller Element)

用法： `previousSmaller(a)[i]` 是左側最近且嚴格較小值的索引，不存在為 -1。時間 $O(n)$。常用於直方圖與區間邊界；有重複值時，要留意嚴格 / 非嚴格比較。

```
vector<int> previousSmaller(const vector<ll>& a) {
    vector<int> st,ans(a.size(),-1);
    for (int i=0;i<(int)a.size();++i) {
        while (!st.empty() && a[st.back()]>=a[i])
            st.pop_back();
        if (!st.empty()) ans[i]=st.back();
        st.push_back(i);
    }
    return ans;
}
```

16. Additional Graph and Tree Algorithms

BFS / Multi-source BFS

用法： `bfs(g,{s})` 回傳無權圖的最短邊數，不可達為 -1；多源使用 `{s1,s2}`。g[u] 存鄰點。時間 $O(V+E)$；格子圖連接合法相鄰格，無向邊需加入兩個方向。

```
vector<int> bfs(const vector<vector<int>>& g,
                const vector<int>& sources) {
    vector<int> d(g.size(),-1); queue<int> q;
    for (int s:sources) if (d[s]==-1) {
        d[s]=0; q.push(s);
    }
    while (!q.empty()) {
        int u=q.front(); q.pop();
        for (int v:g[u]) if (d[v]==-1) {
            d[v]=d[u]+1; q.push(v);
        }
    }
    return d;
}
```

0--1 BFS

用法： `zeroOneBFS(g,s)`，g[u] 存 `{v,w}`。**權重只能是 0 或 1**；不可達為 BOOK_INF。時間 $O(V+E)$。0 權放 deque 前端，1 權放後端；其他非負權使用 Dijkstra。

```
vector<ll> zeroOneBFS(const Graph01& g,int s) {
    vector<ll> d(g.size(),BOOK_INF);
    deque<pair<ll,int>> q;
    d[s]=0; q.push_front({0,s});
    while (!q.empty()) {
        ll du=q.front().first;
        int u=q.front().second; q.pop_front();
        if (du!=d[u]) continue;
        for (auto e:g[u]) {
            int v=e.first; ll w=e.second;
            if (d[v]<=du+w) continue;
            d[v]=du+w;
            if (w==0) q.push_front({d[v],v});
            else q.push_back({d[v],v});
        }
    }
    return d;
}
```

Bridges and Articulation Points

用法： `Lowlink b(n); b.addEdge(u,v); b.run()`；`b.bridges` 存橋的 edge ID，`b.cut[u]` 標記割點。時間 $O(V+E)$，支援不連通圖、平行邊與自環。以 edge ID 跳過父邊，才能正確處理平行邊。DFS 根至少有兩個 DFS 子樹才是割點。長鏈注意遞迴深度；每個物件只呼叫 run 一次。

```
struct Lowlink {
    int timer=0,edgeCount=0;
    vector<vector<pair<int,int>>> g;
    vector<int> tin,low,bridges;
    vector<char> cut;
    Lowlink(int n):g(n),tin(n,-1),low(n),cut(n) {}
    void addEdge(int u,int v) {
        g[u].push_back({v,edgeCount});
        g[v].push_back({u,edgeCount++});
    }
    void dfs(int u,int parentEdge) {
        tin[u]=low[u]=timer++; int children=0;
        for (auto e:g[u]) {
            int v=e.first,id=e.second;
            if (id==parentEdge) continue;
```

```
            if (tin[v]!=-1)
                low[u]=min(low[u],tin[v]);
            else {
                ++children; dfs(v,id);
                low[u]=min(low[u],low[v]);
                if (low[v]>tin[u])
                    bridges.push_back(id);
                if (parentEdge!=-1 && low[v]>=tin[u])
                    cut[u]=1;
            }
        }
        if (parentEdge==-1 && children>1) cut[u]=1;
    }
    void run() {
        for (int u=0;u<(int)g.size();++u)
            if (tin[u]==-1) dfs(u,-1);
    }
};
```

2-SAT (Boolean Constraints)

依賴： 第 8 章 Tarjan。`TwoSAT s(n);` `s.addOr(i,true,j,false)` 加入「xi 為真或 xj 為假」。`s.force(i,true)` 強制 xi 為真；`s.solve()` 成立後讀取 `s.value[i]`。時間 $O(V+E)$。蘊含 a -> b 寫成「非 a 或 b」。變數編號 0..n-1；答案比較方向對應原本 Tarjan 的 SCC 編號。

```
struct TwoSAT {
    int n; vector<vector<int>> g;
    vector<int> value;
    TwoSAT(int n):n(n),g(2*n),value(n) {}
    int literal(int i,bool val) { return 2*i+val; }
    void addOr(int i,bool a,int j,bool b) {
        int u=literal(i,a),v=literal(j,b);
        g[u^1].push_back(v); g[v^1].push_back(u);
    }
    void force(int i,bool a) { addOr(i,a,i,a); }
    bool solve() {
        Tarjan s(2*n); s.adj=g; s.solve();
        for (int i=0;i<n;++i) {
            if (s.comp[2*i]==s.comp[2*i+1]) return
                false;
            value[i]=s.comp[2*i+1]<s.comp[2*i];
        }
        return true;
    }
};
```

Dinic (Maximum Flow / Minimum Cut)

用法： `Dinic f(n); f.addEdge(u,v,cap); f.maxflow(s,t);` s != t，容量須為非負整數。一般時間界 $O(V^2E)$。addEdge 回傳正向 edge ID，流量為反向邊的 cap。最大流後，`f.mincut(s)` 標記殘量圖中 s 可到的點；原圖跨出此集合的邊構成最小割。二分圖匹配：源到左、左到右、右到匯皆設容量 1。點容量用 in/out 拆點；長增廣路需注意堆疊深度。

```
struct Dinic {
    struct E { int to; ll cap; };
    vector<E> e; vector<vector<int>> g;
    vector<int> level,it;
    Dinic(int n):g(n),level(n),it(n) {}
    int addEdge(int u,int v,ll cap) {
        int id=e.size();
        e.push_back({v,cap}); e.push_back({u,0});
        g[u].push_back(id); g[v].push_back(id^1);
        return id;
    }
    bool bfsLevel(int s,int t) {
        fill(level.begin(),level.end(),-1);
        queue<int> q; level[s]=0; q.push(s);
        while (!q.empty()) {
            int u=q.front(); q.pop();
            for (int id:g[u]) if (e[id].cap>0
                && level[e[id].to]==-1) {
                level[e[id].to]=level[u]+1;
                q.push(e[id].to);
            }
        }
        return level[t]!=-1;
    }
    ll dfs(int u,int t,ll pushed) {
        if (u==t || !pushed) return pushed;
        for (int& i=it[u];i<(int)g[u].size();++i) {
            int id=g[u][i],v=e[id].to;
            if (!e[id].cap || level[v]!=level[u]+1)
                continue;
            ll f=dfs(v,t,min(pushed,e[id].cap));
            if (!f) continue;
            e[id].cap-=f; e[id^1].cap+=f;
            return f;
        }
        return 0;
    }
    ll maxflow(int s,int t) {
        assert(s!=t); ll ans=0;
        while (bfsLevel(s,t)) {
            fill(it.begin(),it.end(),0);
            while (ll f=dfs(s,t,BOOK_INF)) ans+=f;
        }
        return ans;
    }
    vector<char> mincut(int s) const {
        vector<char> vis(g.size()); queue<int> q;
        vis[s]=1; q.push(s);
        while (!q.empty()) {
            int u=q.front(); q.pop();
            for (int id:g[u]) if (e[id].cap>0
                && !vis[e[id].to]) {
                vis[e[id].to]=1; q.push(e[id].to);
            }
        }
        return vis;
    }
};
```

Heavy-Light Decomposition (Tree Path Queries)

用法： `HLD h(g,root)`，g 為連通無向樹。點值放在 `base[h.pos[u]+1]`。**依賴：** 第 13 章 SegTree，以 `st.build(base,1,1,n)` 建立。路徑加值：`h.path(u,v, [&] (int l,int r) {st.update(l+1,r+1,x);});`。路徑和：對同樣的區段累加 `st.query(l+1,r+1)`。剖分 $O(n)$，每次操作 $O(\log^2 n)$。子樹區間為 `[pos[u],pos[u]+sz[u]-1]`。邊值存較深端，path 最後參數傳 true 以排除 LCA。區段方向不保證一致，適用 sum/min/max；字串串接等運算需另外維護方向。

```
struct HLD {
    vector<int> parent,depth,sz,heavy,head,pos;
    HLD(const vector<vector<int>>& g,int root=0) {
        int n=g.size();
        parent.assign(n,-1); depth.resize(n);
        sz.assign(n,1); heavy.assign(n,-1);
        head.resize(n); pos.resize(n);
        vector<int> order{root}; parent[root]=root;
        for (int i=0;i<(int)order.size();++i) {
            int u=order[i];
            for (int v:g[u]) if (v!=parent[u]) {
                parent[v]=u; depth[v]=depth[u]+1;
                order.push_back(v);
            }
```

```
        }
        for (int i=n-1;i>=0;--i) {
            int u=order[i];
            for (int v:g[u]) if (parent[v]==u && v!
             ↪ =u) {
                sz[u]+=sz[v];
                if (heavy[u]==-1 ||
                 ↪ sz[v]>sz[heavy[u]])
                    heavy[u]=v;
            }
        }
        int timer=0;
        vector<pair<int,int>> pending{{root,root}};
        while (!pending.empty()) {
            auto p=pending.back();
             ↪ pending.pop_back();
            for (int u=p.first;u!=-1;u=heavy[u]) {
                head[u]=p.second; pos[u]=timer++;
                for (int v:g[u]) if (v!=parent[u]
                    && v!=heavy[u])
                     ↪ pending.push_back({v,v});
            }
        }
    }
    template<class F>
    void path(int u,int v,F use,bool edges=false)
     ↪ const {
        while (head[u]!=head[v]) {
            if (depth[head[u]]<depth[head[v]])
             ↪ swap(u,v);
            use(pos[head[u]],pos[u]);
            u=parent[head[u]];
        }
        if (depth[u]>depth[v]) swap(u,v);
        int l=pos[u]+edges;
        if (l<=pos[v]) use(l,pos[v]);
    }
};
```

17. Additional Dynamic Programming

LIS Reconstruction

用法： `auto ids=lisIndices(a)` 回傳一組最長嚴格遞增子序列的原始索引。時間 $O(n \log n)$。只求長度可沿用第 9 章；非遞減序列改 upper_bound。tail 陣列本身不是還原答案。

```
vector<int> lisIndices(const vector<ll>& a) {
    vector<ll> tail;
    vector<int> at,prev(a.size(),-1);
    for (int i=0;i<(int)a.size();++i) {
        int k=lower_bound(tail.begin(),tail.end(),a[
         ↪ i])
            -tail.begin();
        if (k) prev[i]=at[k-1];
        if (k==(int)tail.size()) {
            tail.push_back(a[i]); at.push_back(i);
        } else { tail[k]=a[i]; at[k]=i; }
    }
    vector<int> ans;
    if (at.empty()) return ans;
    for (int i=at.back();i!=-1;i=prev[i])
     ↪ ans.push_back(i);
    reverse(ans.begin(),ans.end()); return ans;
}
```

0/1 Knapsack and Unbounded Knapsack

用法： `knapsack01(W,items)`，items 存 `{weight,value}`；最大化總重量不超過 W 的價值，允許不選。0/1 背包容量倒序；`knapsackUnlimited` 容量正序。重量須正整數；時間 $O(nW)$、空間 $O(W)$。若要恰好裝滿，除 dp[0] 外初始設 -BOOK_INF，轉移前檢查可達。多重背包可拆成 1、2、4、剩餘件數。

```
ll knapsack01(int W,const vector<pair<int,ll>>&
 ↪ items) {
    vector<ll> dp(W+1);
    for (auto item:items) {
        int w=item.first; ll v=item.second;
        for (int c=W;c>=w;--c)
            dp[c]=max(dp[c],dp[c-w]+v);
    }
    return dp[W];
}
ll knapsackUnlimited(int W,
    const vector<pair<int,ll>>& items) {
    vector<ll> dp(W+1);
    for (auto item:items) {
        int w=item.first; ll v=item.second;
        for (int c=w;c<=W;++c)
            dp[c]=max(dp[c],dp[c-w]+v);
    }
    return dp[W];
}
```

Longest Common Subsequence / Edit Distance

用法： `lcs(a,b)` 回傳最長共同子序列長度；`editDistance(a,b)` 的插入、刪除、替換成本皆為 1。時間 $O(nm)$，空間 $O(m)$。此版僅回傳數值；若要還原解，需記錄完整 DP 表。

```
int lcs(const string& a,const string& b) {
    vector<int> dp(b.size()+1);
    for (char x:a) {
        int diagonal=0;
        for (int j=1;j<=(int)b.size();++j) {
            int old=dp[j];
            if (x==b[j-1]) dp[j]=diagonal+1;
            else dp[j]=max(dp[j],dp[j-1]);
            diagonal=old;
        }
    }
    return dp.back();
}
int editDistance(const string& a,const string& b) {
    vector<int> dp(b.size()+1);
    iota(dp.begin(),dp.end(),0);
    for (int i=1;i<=(int)a.size();++i) {
        int diagonal=dp[0]; dp[0]=i;
        for (int j=1;j<=(int)b.size();++j) {
            int old=dp[j];
            dp[j]=min({dp[j]+1,dp[j-1]+1,
                       diagonal+(a[i-1]!=b[j-1])});
            diagonal=old;
        }
    }
    return dp.back();
}
```

Bitmask DP (Traveling Salesman)

用法： `tsp(c)` 求從 0 出發、每點恰走一次、回 0 的最小成本。無邊設 c[u][v]=BOOK_INF。時間 $O(n^2 2^n)$，空間 $O(n2^n)$。n <= 20，但 n=20 的 ll 表約需 160 MiB，先看記憶體限制。`dp[mask][u]` 表示已走過 mask、最後停在 u 的最佳成本；此版不還原路徑。

```
ll tsp(const vector<vector<ll>>& c) {
    int n=c.size(); if (n<=1) return 0;
    assert(n<=20); int all=1<<n;
    vector<ll> dp((size_t)all*n,BOOK_INF);
    dp[n]=0; // mask=1, u=0
    for (int mask=1;mask<all;mask+=2)
```

```
        for (int u=0;u<n;++u) {
            ll cur=dp[(size_t)mask*n+u];
            if (cur==BOOK_INF) continue;
            for (int v=0;v<n;++v) if (!(mask>>v&1)
                && c[u][v]!=BOOK_INF) {
                ll& to=dp[(size_t)(mask|(1<<v))*n+v];
                to=min(to,cur+c[u][v]);
            }
        }
    ll ans=BOOK_INF;
    for (int u=1;u<n;++u) if (c[u][0]!=BOOK_INF) {
        ll cur=dp[(size_t)(all-1)*n+u];
        if (cur!=BOOK_INF) ans=min(ans,cur+c[u][0]);
    }
    return ans;
}
```

Digit DP (No Equal Adjacent Digits)

用法： `countNoAdjacent(N)` 計算 [0,N] 中相鄰十進位數字不相同的整數，包含 0；N < 0 回傳 0。區間 [L,R] 用 f(R)-f(L-1)。時間約 $O(\log N \cdot 110)$；prev=10 代表前導零，不算真正數字。依題目調整狀態；只記憶化 tight=false 的狀態，避免混用不同上界。

```
ll countNoAdjacent(ll N) {
    if (N<0) return 0;
    string s=to_string(N);
    ll memo[20][11]; memset(memo,-1,sizeof(memo));
    function<ll(int,int,bool)> dfs=
        [&](int pos,int prev,bool tight)->ll {
        if (pos==(int)s.size()) return 1;
        if (!tight && memo[pos][prev]!=-1)
            return memo[pos][prev];
        int limit=tight?s[pos]-'0':9; ll ans=0;
        for (int d=0;d<=limit;++d) {
            if (d==prev) continue;
            int nxt=(prev==10 && d==0)?10:d;
            ans+=dfs(pos+1,nxt,tight && d==limit);
        }
        if (!tight) memo[pos][prev]=ans;
        return ans;
    };
    return dfs(0,10,true);
}
```

18. Additional String Algorithms

Z Algorithm

用法： `zFunction(s)[i]` 為 s 與 s[i..] 的最長共同前綴長度，z[0]=n。時間 $O(n)$。匹配可對 pattern + 分隔符 + text 計算 Z，分隔符不可出現在任一輸入字串內。

```
vector<int> zFunction(const string& s) {
    int n=s.size(); vector<int> z(n);
    for (int i=1,l=0,r=0;i<n;++i) {
        if (i<r) z[i]=min(r-i,z[i-l]);
        while (i+z[i]<n && s[z[i]]==s[i+z[i]]) ++z[i];
        if (i+z[i]>r) { l=i; r=i+z[i]; }
    }
    if (n) z[0]=n;
    return z;
}
```

Manacher (Palindrome Radii)

用法： `auto d=manacher(s)`；d.first[i] 為含中心的奇回文半徑，長度為 2d-1。d.second[i] 為 i-1 與 i 中間的偶回文半徑，長度為 2d。時間與空間皆 $O(n)$。全部半徑相加為回文子字串總數；取對應長度的最大值可得最長回文。

```
pair<vector<int>,vector<int>> manacher(const string& s) {
    int n=s.size(); vector<int> odd(n),even(n);
    for (int i=0,l=0,r=-1;i<n;++i) {
        int k=i>r?1:min(odd[l+r-i],r-i+1);
        while (i-k>=0 && i+k<n && s[i-k]==s[i+k]) ++k;
        odd[i]=k--;
        if (i+k>r) { l=i-k; r=i+k; }
    }
    for (int i=0,l=0,r=-1;i<n;++i) {
        int k=i>r?0:min(even[l+r-i+1],r-i+1);
        while (i-k-1>=0 && i+k<n
               && s[i-k-1]==s[i+k]) ++k;
        even[i]=k--;
        if (i+k>r) { l=i-k-1; r=i+k; }
    }
    return {odd,even};
}
```

19. Additional Number Theory and Algebra

Linear Sieve / Smallest Prime Factor

用法： `LinearSieve s(N)`；`s.primes` 為質數表，x >= 2 時 `s.lp[x]==x` 表示質數。`s.factor(x)` 可在 $O(\log x)$ 分解 1 <= x <= N。建立 $O(N)$；lp 陣列約需 4N bytes。

```
struct LinearSieve {
    vector<int> lp,primes;
    LinearSieve(int n):lp(n+1) {
        for (int i=2;i<=n;++i) {
            if (!lp[i]) { lp[i]=i; primes.push_back(i); }
            for (int p:primes) {
                if (p>lp[i] || p>n/i) break;
                lp[i*p]=p;
            }
        }
    }
    vector<pair<int,int>> factor(int x) const {
        vector<pair<int,int>> ans;
        while (x>1) {
            int p=lp[x],e=0;
            do { x/=p; ++e; } while (x>1 && lp[x]==p);
            ans.push_back({p,e});
        }
        return ans;
    }
};
```

Extended GCD / General Modular Inverse

用法： `extGcd(a,b,x,y)` 回傳 gcd，並滿足 ax+by=gcd，輸入 a、b 須非負。`inverse(a,m)` 回傳模逆元，無逆元為 -1；m > 1，不必為質數。時間 $O(\log m)$。解 ax+by=c 時需 gcd 整除 c，再將 x、y 乘 c/gcd；注意乘法溢位。

```
ll normalize(ll a,ll m) {
    a%=m; return a<0?a+m:a;
}
```

```
ll extGcd(ll a,ll b,ll& x,ll& y) {
    if (!b) { x=1; y=0; return a; }
    ll u,v,g=extGcd(b,a%b,u,v);
    x=v; y=u-(a/b)*v; return g;
```

```
}
ll inverse(ll a,ll m) {
    ll x,y,g=extGcd(normalize(a,m),m,x,y);
    return g==1?normalize(x,m):-1;
}
```

Safe 64-bit Modular Multiplication / Miller--Rabin

用法： `isPrime64(n)` 對整個 unsigned 64-bit 範圍做確定性質數判定，不做因數分解。 以倍增模乘避免溢位，Miller--Rabin 約 $O(\log^2 n)$；小數值仍可沿用第 2 章試除。 若 64-bit GCC 支援，可把 mulMod64 改成 `((__uint128_t)a*b)%m` 加速。 固定測試底數依據 CP-Algorithms。

```
ull addMod64(ull a,ull b,ull m) { // a,b < m
    return a>=m-b?a-(m-b):a+b;
}
ull mulMod64(ull a,ull b,ull m) {
    a%=m; ull ans=0;
    while (b) {
        if (b&1) ans=addMod64(ans,a,m);
        a=addMod64(a,a,m); b>>=1;
    }
    return ans;
}
ull powMod64(ull a,ull e,ull m) {
    ull ans=1%m;
    while (e) {
        if (e&1) ans=mulMod64(ans,a,m);
        a=mulMod64(a,a,m); e>>=1;
    }
    return ans;
}
bool isPrime64(ull n) {
    if (n<2) return false;
    for (ull p:{2ULL,3ULL,5ULL,7ULL,11ULL,13ULL,
                17ULL,19ULL,23ULL,29ULL,31ULL,37ULL})
        if (n%p==0) return n==p;
    ull d=n-1; int s=0;
    while (!(d&1)) { d>>=1; ++s; }
    for (ull a:{2ULL,325ULL,9375ULL,28178ULL,
                450775ULL,9780504ULL,1795265022ULL})
        if (a%n==0) continue;
        ull x=powMod64(a,d,n);
        if (x==1 || x==n-1) continue;
        bool pass=false;
        for (int r=1;r<s;++r) {
            x=mulMod64(x,x,n);
            if (x==n-1) { pass=true; break; }
        }
        if (!pass) return false;
    }
    return true;
}
```

Generalized Chinese Remainder Theorem

用法： `crtMerge(a,m,b,n)` 合併 x=a (mod m)、x=b (mod n)，回傳最小非負解與合併模數。 模數須為正，不必互質。第二項為 0 表示無解，-1 表示合併模數超出 ll。 **依賴：** normalize、extGcd、mulMod64。 從 `{0,1}` 起逐條合併；(2,6,5,9) 會得到 (14,18)。

```
pair<ll,ll> crtMerge(ll a,ll m,ll b,ll n) {
    a=normalize(a,m); b=normalize(b,n);
    ll x,y,g=extGcd(m,n,x,y),delta=b-a;
    if (delta%g) return {0,0};
    ll reduced=n/g;
    if (m>numeric_limits<ll>::max()/reduced)
        return {0,-1};
    ll k=mulMod64(normalize(delta/g,reduced),
                  normalize(x,reduced),reduced);
    ll mod=m*reduced;
    return {a+m*k,mod};
}
```

Matrix Exponentiation

用法： `matrixPow(A,k,mod)` 計算方陣 A 的 k 次方，k >= 0，模數不超過 2*10^9。 輸入元素先 normalize。時間 $O(d^3 \log k)$；Fibonacci 為 `{{1,1},{1,0}}` 的 n 次方 [0][1]。

```
using Matrix=vector<vector<ll>>;
Matrix matrixMul(const Matrix& a,const Matrix& b,ll
    mod) {
    int n=a.size(); Matrix c(n,vector<ll>(n));
    for (int i=0;i<n;++i)
        for (int k=0;k<n;++k) if (a[i][k])
            for (int j=0;j<n;++j)
                c[i][j]=(c[i][j]+a[i][k]*b[k][j])
    %mod;
    return c;
}
Matrix matrixPow(Matrix a,ll k,ll mod=1000000007) {
    int n=a.size(); Matrix r(n,vector<ll>(n));
    for (int i=0;i<n;++i) r[i][i]=1%mod;
    while (k) {
        if (k&1) r=matrixMul(r,a,mod);
        a=matrixMul(a,a,mod); k>>=1;
    }
    return r;
}
```

XOR Linear Basis

用法： `XorBasis b; b.insert(x); b.maxXor()` 求任意子集合 XOR 最大值；`contains(x)` 判斷可否組出。 每次 $O(64)$，支援 ull 全範圍；insert 回傳 false 表示線性相依，rank 為基底維度。

```
struct XorBasis {
    array<ull,64> b{}; int rank=0;
    bool insert(ull x) {
        for (int k=63;k>=0;--k) if (x>>k&1) {
            if (!b[k]) { b[k]=x; ++rank; return true;
            }
            x^=b[k];
        }
        return false;
    }
    bool contains(ull x) const {
        for (int k=63;k>=0;--k)
            if (x>>k&1) x^=b[k];
        return x==0;
    }
    ull maxXor(ull x=0) const {
        for (int k=63;k>=0;--k) x=max(x,x^b[k]);
        return x;
    }
};
```

20. Additional Computational Geometry

Integer Cross Product / Segment Intersection

整數 `IPoint` 與第 4 章浮點 Point 分開使用。 `cross(a,b,c)` 正為逆時針，負為順時針；`segmentsIntersect(a,b,c,d)` 包含端點接觸與共線重疊。 時間 $O(1)$。座標絕對值不超過 10^9 時，單次叉積可放入 ll；面積累加仍可能溢位。

```cpp
struct IPoint {
    ll x,y;
    bool operator<(const IPoint& p) const {
        return tie(x,y)<tie(p.x,p.y);
    }
    bool operator==(const IPoint& p) const {
        return x==p.x && y==p.y;
    }
};
ll cross(IPoint a,IPoint b,IPoint c) {
    return (b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x);
}
ll dot(IPoint a,IPoint b,IPoint c) {
    return (b.x-a.x)*(c.x-a.x)+(b.y-a.y)*(c.y-a.y);
}
bool onSegment(IPoint a,IPoint b,IPoint p) {
    return cross(a,b,p)==0
        && min(a.x,b.x)<=p.x && p.x<=max(a.x,b.x)
        && min(a.y,b.y)<=p.y && p.y<=max(a.y,b.y);
}
int sign(ll x) { return (x>0)-(x<0); }
bool segmentsIntersect(IPoint a,IPoint b,IPoint
 ↪ c,IPoint d) {
    if (onSegment(a,b,c) || onSegment(a,b,d)
        || onSegment(c,d,a) || onSegment(c,d,b))
         ↪ return true;
    return sign(cross(a,b,c))*sign(cross(a,b,d))<0
        && sign(cross(c,d,a))*sign(cross(c,d,b))<0;
}
```

Convex Hull (Andrew Monotone Chain)

用法： `convexHull(points)` 接受 IPoint，回傳逆時針凸包，不重複首尾頂點。時間 $O(n \log n)$；去重並排除邊上的中間共線點，全部共線時只留兩端。**依賴：** IPoint、cross。若要保留共線點，需另外處理全部共線的情形。

```cpp
vector<IPoint> convexHull(vector<IPoint> p) {
    sort(p.begin(),p.end());
    p.erase(unique(p.begin(),p.end()),p.end());
    if (p.size()<=1) return p;
    vector<IPoint> lo,hi;
    for (IPoint x:p) {
        while (lo.size()>=2
            && cross(lo[lo.size()-2],lo.back(),x)<=0)
            lo.pop_back();
        lo.push_back(x);
    }
    for (int i=(int)p.size()-1;i>=0;--i) {
        IPoint x=p[i];
        while (hi.size()>=2
            && cross(hi[hi.size()-2],hi.back(),x)<=0)
            hi.pop_back();
        hi.push_back(x);
    }
    lo.pop_back(); hi.pop_back();
    lo.insert(lo.end(),hi.begin(),hi.end()); return
 ↪ lo;
}
```

Polygon Area / Point in Polygon

用法： `polygonArea2(p)` 回傳簡單多邊形的兩倍面積；`pointInPolygon(p,q)` 回傳 0 外部、1 內部、2 邊界。時間 $O(n)$，可處理凹多邊形，不用重複首點。依賴 IPoint、cross、onSegment。格點多邊形的 Pick 定理為 $A = I + B/2 - 1$，每條邊貢獻 gcd(|dx|,|dy|) 個邊界格點。

```cpp
ll polygonArea2(const vector<IPoint>& p) {
    ll sum=0; int n=p.size();
    for (int i=0;i<n;++i) {
        IPoint a=p[i],b=p[(i+1)%n];
        sum+=a.x*b.y-a.y*b.x;
    }
    return abs(sum);
}
int pointInPolygon(const vector<IPoint>& p,IPoint q)
 ↪ {
    bool inside=false; int n=p.size();
    for (int i=0;i<n;++i) {
        IPoint a=p[i],b=p[(i+1)%n];
        if (onSegment(a,b,q)) return 2;
        if ((a.y>q.y)!=(b.y>q.y)) {
            ll c=cross(a,b,q);
            if ((c>0)==(b.y>a.y)) inside=!inside;
        }
    }
    return inside?1:0;
}
```

21. Offline Queries and Meet in the Middle

Mo's Algorithm (Distinct Values in a Range)

用法： `moDistinct(a,queries)` 接受 `{l,r}` 查詢，依輸入順序回傳區間不同值個數。a 先壓縮到 [0,n)，不支援修改；約需 $O((n+q)\sqrt{n})$ 次增刪、$O(n+q)$ 空間。其他統計量可改 add/del 維護；若有較簡單的離線 BIT 解法，優先考慮。

```cpp
vector<int> moDistinct(const vector<int>& a,
                       const vector<pair<int,int>>&
                        ↪ ranges) {
    int n=a.size(),q=ranges.size();
    int block=max(1,(int)sqrt(n));
    vector<int> ids(q),freq(n),ans(q);
    iota(ids.begin(),ids.end(),0);
    sort(ids.begin(),ids.end(),[&](int i,int j) {
        int x=ranges[i].first/block,y=ranges[j].
         ↪ first/block;
        if (x!=y) return x<y;
        return x&1?ranges[i].second>ranges[j].second
                  :ranges[i].second<ranges[j].second;
    });
    int l=0,r=-1,distinct=0;
    auto add=[&](int i) { if (freq[a[i]]++==0) +
     ↪ +distinct; };
    auto del=[&](int i) { if (--freq[a[i]]==0)
     ↪ --distinct; };
    for (int id:ids) {
        int L=ranges[id].first,R=ranges[id].second;
        while (l>L) add(--l);
        while (r<R) add(++r);
        while (l<L) del(l++);
        while (r>R) del(r--);
        ans[id]=distinct;
    }
    return ans;
}
```

Meet in the Middle (Subset Sum Count)

用法： `subsetSumCount(a,target)` 計算和為 target 的子集合數，包含空集合，支援負值。n <= 40；拆成兩半枚舉，再以二分搜尋配對。時間 $O(n2^{n/2})$，空間 $O(2^{n/2})$。

```cpp
ll subsetSumCount(const vector<ll>& a,ll target) {
    int mid=a.size()/2;
    auto enumerate=[&](int l,int r) {
        vector<ll> s{0};
        for (int i=l;i<r;++i) {
```

```
            int old=s.size();
            for (int j=0;j<old;++j) s.push_back(s[j]
             ↪  +a[i]);
        }
        return s;
    };
    auto left=enumerate(0,mid);
    auto right=enumerate(mid,a.size());
    sort(right.begin(),right.end()); ll ans=0;
    for (ll x:left) {
        auto p=equal_range(right.begin(),right.end
         ↪  (),target-x);
        ans+=p.second-p.first;
    }
    return ans;
}
```

Submission Checklist

- 原有模板索引規則不同，使用前先看註解；BIT 不可 add(0)，原本的 LCA 為 1-based。
- 第 6 章 Phi 在 x=1 回傳 0，標準值應為 1；modInverse 要求質數模數且餘數非零。
- 第 4 章最近點對每層排序 strip，實際為 $O(n \log^2 n)$；至少需兩個輸入點。
- 第 8 章 Floyd 更新前須確認兩段皆可達，避免 INF 加負權形成假距離。
- 第 11 章 AC 的時間需加上輸出量；複製 out 列表也可能占用大量記憶體。
- 大乘積先轉 ll；加值前檢查可達；多筆測資重設狀態；長鏈注意遞迴堆疊。
- 以小型隨機資料和暴力解對拍，另試空集合、全相等、全負值、平行邊與不連通圖。